MATERIAL DIDÁCTICO · ACCESO A DATOS

TUTORIAL RESUELTO 04 · UNIDAD 02

MongoDB y documentos con Java

Servidor local, driver síncrono, documentos anidados, CRUD y comprobaciones reproducibles.

Objetivo y resultado

Construir una aplicación que guarda clubes como documentos BSON en MongoDB, sin tablas ni claves externas SQL. Incluiremos jugadores como subdocumentos y realizaremos altas reproducibles, búsqueda, modificación y borrado.

Paso 1. Encender MongoDB

Abre una terminal dentro de la carpeta Codigo/Infraestructura/mongo y ejecuta:

TERMINAL

```
docker compose up -d
docker compose ps
docker compose exec mongo mongosh
```

En la consola interactiva puedes comprobar inicialmente que no hay documentos. No necesitas crear colecciones a mano.

MONGOSH

```
use ad_ud2
show collections
db.club.find()
exit
```

1.1. compose.yaml de MongoDB

Ruta: **Codigo/Infraestructura/mongo/compose.yaml**

ARCHIVO COMPLETO

```
services:
  mongo:
    image: mongo:7.0
    ports:
      - "127.0.0.1:27018:27017"
    volumes:
      - mongo_ad_ud2:/data/db
volumes:
  mongo_ad_ud2:
```

El puerto publicado 27018 evita entrar en conflicto con otro MongoDB que ya escuche en 27017. La base no tiene autenticación y solo se utiliza en local.

Paso 2. Crear un proyecto Maven Java

Abre la carpeta **Codigo/T04_MongoDB_Documental** en el IDE. El archivo pom.xml incorpora el driver síncrono oficial.

TERMINAL

```
cd Codigo/T04_MongoDB_Documental
mvn clean compile exec:java
```

2.1. Dependencias Maven

Ruta: **Codigo/T04_MongoDB_Documental/pom.xml**

ARCHIVO COMPLETO

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>es.curso.ad</groupId>
  <artifactId>t04-mongodb</artifactId>
  <version>1.0-SNAPSHOT</version>
  <properties>
    <maven.compiler.release>17</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.mongodb</groupId>
      <artifactId>mongodb-driver-sync</artifactId>
      <version>5.4.0</version>
    </dependency>
  </dependencies>
  <build>
    <plugins>
      <plugin>
        <groupId>org.codehaus.mojo</groupId>
        <artifactId>exec-maven-plugin</artifactId>
        <version>3.5.0</version>
        <configuration><mainClass>es.curso.ad.t04.Main</mainClass></configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

Paso 3. Programar el repositorio documental

MongoClients crea la conexión; getDatabase y getCollection seleccionan la colección. Se utiliza replaceOne con upsert para cargar los dos clubes sin duplicados, y se implementan los métodos listar, buscar, cambiarCiudad y borrar. La clase es AutoCloseable para garantizar el cierre del cliente.

3.1. DAO documental completo

Ruta: **Codigo/T04_MongoDB_Documental/src/main/java/es/curso/ad/t04/ClubDocumentalDao.java**

ARCHIVO COMPLETO

```
package es.curso.ad.t04;

import com.mongodb.client.MongoClient;
import com.mongodb.client.MongoClients;
import com.mongodb.client.MongoCollection;
import com.mongodb.client.MongoDatabase;
import com.mongodb.client.model.ReplaceOptions;
import com.mongodb.client.model.Sorts;
```

```
import java.util.ArrayList;
import java.util.List;
import org.bson.Document;
import static com.mongodb.client.model.Filters.eq;
import static com.mongodb.client.model.Updates.set;

public class ClubDocumentalDao implements AutoCloseable {
    private final MongoClient cliente;
    private final MongoCollection<Document> clubes;

    public ClubDocumentalDao() {
        cliente = MongoClients.create("mongodb://localhost:27018");
        MongoDatabase db = cliente.getDatabase("ad_ud2");
        clubes = db.getCollection("club");
    }

    public void guardarEjemplos() {
        Document granada = new Document("_id", 1)
                .append("nombre", "Granada CF")
                .append("ciudad", "Granada")
                .append("jugadores", List.of(
                        new Document("nombre", "Lucía").append("dorsal", 10),
                        new Document("nombre", "Pablo").append("dorsal", 7)));
        Document malaga = new Document("_id", 2)
                .append("nombre", "Málaga CF")
                .append("ciudad", "Málaga")
                .append("jugadores", List.of(
                        new Document("nombre", "Sara").append("dorsal", 9)));
        ReplaceOptions opciones = new ReplaceOptions().upsert(true);
        clubes.replaceOne(eq("_id", 1), granada, opciones);
        clubes.replaceOne(eq("_id", 2), malaga, opciones);
    }

    public List<Document> listar() {
        return clubes.find().sort(Sorts.ascending("_id"))
                .into(new ArrayList<>());
    }

    public Document buscar(int id) {
        return clubes.find(eq("_id", id)).first();
    }

    public boolean cambiarCiudad(int id, String nuevaCiudad) {
        return clubes.updateOne(eq("_id", id), set("ciudad", nuevaCiudad))
                .getMatchedCount() == 1;
    }

    public boolean borrar(int id) {
        return clubes.deleteOne(eq("_id", id)).getDeletedCount() == 1;
    }

    @Override
    public void close() {
        cliente.close();
    }
}
```

3.2. Main.java

Ruta: **Codigo/T04_MongoDB_Documental/src/main/java/es/curso/ad/t04/Main.java**

ARCHIVO COMPLETO

```
package es.curso.ad.t04;

import org.bson.Document;

public class Main {
    public static void main(String[] args) {
        try (ClubDocumentalDao dao = new ClubDocumentalDao()) {
            dao.guardarEjemplos();
            System.out.println("CLUBES EN MONGODB");
            for (Document club : dao.listar()) {
                System.out.println(club.toJson());
            }
            System.out.println("Buscar 1: " + dao.buscar(1).toJson());
            System.out.println("Actualizar 2: " + dao.cambiarCiudad(2, "Málaga"));
            System.out.println("Borrar inexistente: " + dao.borrar(99));
        } catch (RuntimeException e) {
            System.err.println("Error documental: " + e.getMessage());
        }
    }
}
```

La colección se guarda en MongoDB, no en el heap de Java. Repetir la aplicación no duplica los ejemplos porque se conservan los mismos valores _id.

Paso 4. Inspeccionar los documentos desde mongosh

MONGOSH

```
db.club.find().pretty()
db.club.find({ciudad: "Granada"}).pretty()
db.club.find({"jugadores.dorsal": 10}).pretty()
db.club.countDocuments({})
```

Compara los resultados con el SELECT JOIN de MySQL. En MongoDB estamos consultando subdocumentos incrustados: no hay tabla jugador ni FK que la enlace automáticamente.

Paso 5. Probar errores y operaciones

- Busca un ID que no exista y observa que find(...).first() devuelve null.
- Modifica una ciudad con updateOne y consulta después el documento completo.
- Ejecuta borrar sobre el ID 99 y comprueba que getDeletedCount vale cero.
- Repite el programa: hay dos clubes y no cuatro.

> **Diseño documental**
> No incrustes sin criterio listas que puedan crecer indefinidamente. En el ejercicio de la relación decidirás cuándo incrustar y cuándo referenciar.